



If Kepler had a computer

Python simulation of
the solar system

Brescia, 11/12/2023



Let's start simple

In the two-body problem, we want to simulate the dynamics of two objects interacting through gravitational force:

$$\vec{F}_A^G = G \frac{m_A m_B}{|\vec{r}_B - \vec{r}_A|^2} \vec{u}_{BA}$$

$$\vec{F}_B^G = G \frac{m_A m_B}{|\vec{r}_B - \vec{r}_A|^2} \vec{u}_{AB}$$

$$\vec{u}_{BA} = \frac{\vec{r}_B - \vec{r}_A}{|\vec{r}_B - \vec{r}_A|}$$



From Newton to Newton

Thanks to the 2nd law of motion $\vec{F} = m\vec{a}$, we can equate the forces:

$$G \frac{m_A m_B}{|\vec{r}_B - \vec{r}_A|^3} (\vec{r}_B - \vec{r}_A) = m_A \vec{a}_A$$

$$G \frac{m_A m_B}{|\vec{r}_B - \vec{r}_A|^3} (\vec{r}_A - \vec{r}_B) = m_A \vec{a}_B$$



Differential equations 101

We have a system of differential equations! $\vec{a} = \frac{\partial \vec{v}}{\partial t} = \frac{\partial^2 \vec{r}}{\partial t^2}$
so that we can re-write

$$\begin{cases} G \frac{m_A m_B}{|\vec{r}_B - \vec{r}_A|^3} (\vec{r}_B - \vec{r}_A) = m_A \ddot{\vec{r}}_A & \text{for body A,} \\ G \frac{m_A m_B}{|\vec{r}_B - \vec{r}_A|^3} (\vec{r}_A - \vec{r}_B) = m_B \ddot{\vec{r}}_B & \text{for body B.} \end{cases}$$

These are 6 (second-order) **Ordinary Differential Equations**.

$$\frac{\partial \vec{f}}{\partial t} = \dot{\vec{f}}$$



A common trick

We can lower the system's order to 1 to make it easier to solve.

The trade-off is we get more equations in the system:

$$\left\{ \begin{array}{ll} \vec{v}_A = \dot{\vec{r}}_A & \text{for body } A, \\ \vec{v}_B = \dot{\vec{r}}_B & \text{for body } B. \end{array} \right. \longrightarrow \left\{ \begin{array}{l} \dot{\vec{v}}_A = G \frac{m_B}{|\vec{r}_B - \vec{r}_A|^3} (\vec{r}_B - \vec{r}_A), \\ \dot{\vec{v}}_B = G \frac{m_A}{|\vec{r}_B - \vec{r}_A|^3} (\vec{r}_A - \vec{r}_B), \\ \dot{\vec{r}}_A = \vec{v}_A, \\ \dot{\vec{r}}_B = \vec{v}_B. \end{array} \right.$$



Numerical solving with Euler

Any first-order ODE system $y' = f(t, y(t))$ can be simulated with the Euler method, i.e. iterating the following:

1. let $t_{n+1} = t_n + h$;
2. compute $y_{n+1} = y_n + hf(t_n, y_n)$



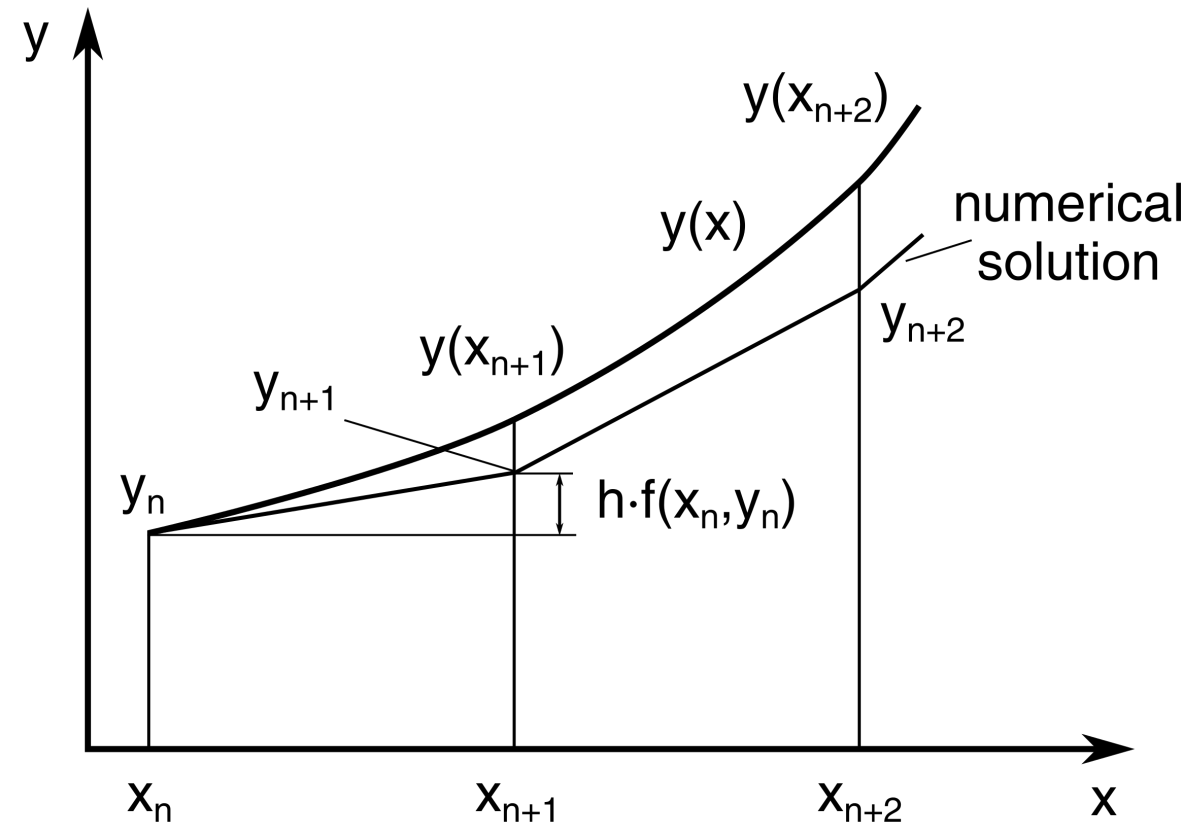
Why does it work?

It creates a **piece-wise linear** reconstruction of the solution by approximating its derivative to the first

$$f(t, y(t)) = y' \approx \frac{y(t+h) - y(t)}{h}$$



$$y(t+h) \approx y(t) + hf(t, y(t))$$





DEMO TIME





About xstream

Visit <https://xtreamers.io/>!

Follow our [LinkedIn page](#) to learn about open positions.

Follow our Head of AI [Emanuele Fabbiani](#).

Connect with our Data Scientist [Gabriele Orlandi](#) and our AI Engineer [Fabio Lipreri](#).





Some Useful Resources

Resources



Enlightening Articles

[Simulate a tiny solar system with Python](#)

[The two-body problem in Python](#)

[Numerical simulation of the planetary motions in the solar system...](#)